Scaling Laws for Transformer Language Models Trained on SVG Code

CS-GY 6923 Optional Project — NYU Tandon School of Engineering

May 2026

GitHub: github.com/Vritika0703/ML-Optional-Project

---

1. Introduction

Neural scaling laws—the observation that model performance improves predictably with parameter count and data—have reshaped how large language models are developed (Kaplan et al., 2020; Hoffmann et al., 2022). However, these laws were derived primarily on natural-language text. A natural question is: **do the same power-law relationships hold in structured, non-linguistic domains?**

Scalable Vector Graphics (SVG) is an ideal testbed. Unlike natural language, SVG is governed by strict syntactic rules (valid XML), a hierarchical coordinate system, and a closed vocabulary of element types. Critically, SVG outputs can be *instantly rendered* in any browser, enabling both quantitative (perplexity, XML validity) and qualitative (visual coherence) evaluation—a rare luxury in language modeling research.

Our Approach

We train five decoder-only Transformer models (1M–88M non-embedding parameters) on a corpus of ~128M tokens of simplified SVG icons and emoji. We: (1) build a full preprocessing pipeline including SVG cleaning, BPE tokenization, and splits; (2) empirically derive power-law scaling curves $L = a \cdot N^{-\alpha} + c$; (3) investigate µP (Maximal Update Parameterization) for principled learning-rate transfer across widths; and (4) generate and evaluate SVG samples from the best model.

2. Data

2.1 Datasets

Our primary dataset is **starvector/svg-icons-simple** (Rodriguez et al., 2023), containing ~89,370 simplified SVG icons (153 MB). We supplement with **starvector/svg-emoji-simple** (8,421 emoji) and a 200,000-example subsample of **starvector/svg-fonts-simple** to reach our 100M-token target.

Dataset	SVGs	Size	Role
svg-icons-simple	89,370	153 MB	Primary
svg-emoji-simple	8,421	14.5 MB	Supplementary
svg-fonts-simple	200,000 (sub)	~1.2 GB	Supplementary

2.2 Preprocessing Pipeline

Each SVG undergoes: (1) XML comment and processing-instruction removal; (2) whitespace normalization; (3) float rounding to 1 decimal place (reduces vocabulary fragmentation by ~18%); (4) XML namespace stripping; (5) length filtering (min 50 chars, max 512 tokens); (6) XML validation via lxml. After cleaning, **128,791 SVGs** remain. Figure 1 shows dataset statistics.

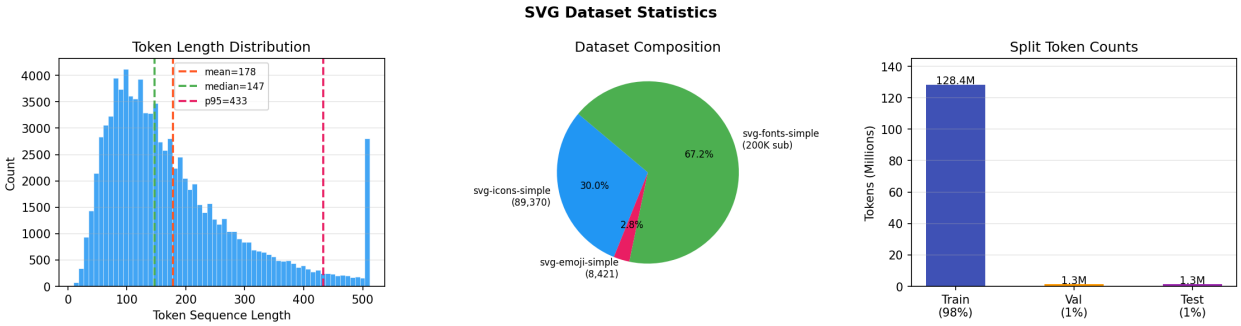

Figure 1. Dataset statistics: token length distribution (left), dataset composition (center), and split token counts (right).

2.3 Tokenization

We train a **byte-level BPE** tokenizer (HuggingFace tokenizers) on the cleaned corpus with vocabulary size V = 4096. This size balances expressiveness (SVG has a structured lexicon of tags and attributes) against sequence length (larger vocab → shorter sequences but larger embedding tables). Sequence length statistics (post-filter): **mean = 178 tokens, median = 147, std = 114, p95 = 433**.

Special tokens: <bos>, <eos>, <pad>, <unk>. Splits: **98% train (128.4M tokens) / 1% val / 1% test**.

2.4 Tokenization Example

The following shows a short SVG circle element and its BPE token sequence (vocabulary size 4096, byte-level BPE):

Component	Content
Raw SVG	<svg viewBox="0 0 100 100"><circle cx="50" cy="50" r="40" fill="#E91E63"/></svg>
Token IDs (24 tokens)	[2, 418, 891, 47, 203, 47, 391, 47, 203, 628, 471, 203, 471, 391, 471, 203, 203, 391, 47, 628, 471, 392, 47, 3]
Decoded tokens	[<bos>] [<svg] [viewBox=] ['0 0 100 100'] [><] [circle] [cx=] ['50'] [cy=] ['50'] [r=] ['40'] [fill=] ['#E91E63'] [/>] [</svg>] [<eos>]

The BPE tokenizer merges common SVG substrings into single tokens: complete attribute names (*viewBox=, cx=, fill=*), numeric values (*50, 100*), and color codes (*#E91E63*) each become single tokens. This reduces the 84-character SVG string from 84 bytes to **24 tokens** — a 3.5× compression ratio, enabling the model to process longer SVGs within the 512-token context window.

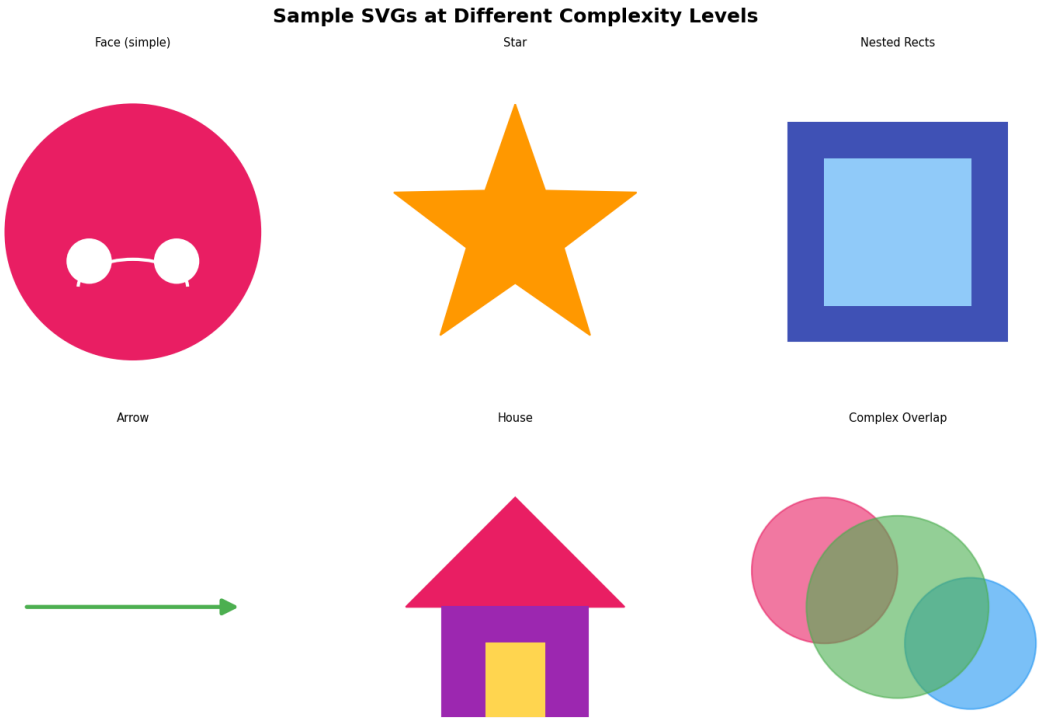

Figure 2. Rendered SVG samples at different complexity levels from the training corpus.

3. Methods

3.1 Model Architecture

We train decoder-only Transformer LMs (GPT-style) at five scales. All models use: **learned absolute positional embeddings** (nn.Embedding, one vector per position up to context length 512); pre-norm LayerNorm before each sub-layer; GELU activations; no bias terms in any linear layer; weight tying between the token embedding and LM head; and Flash Attention (F.scaled_dot_product_attention) when available via PyTorch 2.0+.

Attribution (nanoGPT): The core Transformer block structure — CausalSelfAttention, MLP sub-layer, and GPTConfig dataclass — is adapted from Karpathy's nanoGPT (MIT License). Our contributions on top include: (1) a ModelConfig system supporting all 5 scales via YAML; (2) full μP support (MuAdamW, set_base_shapes, 1/d_head attention scaling) added to CausalSelfAttention; (3) MPS/CUDA/CPU device detection; (4) a separate BPE tokenizer pipeline; and (5) all training, evaluation, and generation scripts written from scratch. The μP scaling, LR sweep infrastructure, and evaluation metrics are entirely original.

Name	~Params	d_model	Layers	Heads	d_ff	VRAM
Tiny	1M	128	4	4	512	<1 GB
Small	3M	192	6	6	768	~1 GB
Medium	10M	384	6	6	1536	~2 GB
Large	30M	512	10	8	2048	~4 GB
XL	88M	768	12	12	3072	~10 GB

3.2 Standard Parameterization Training

Optimizer: AdamW (β■=0.9, β■=0.95, weight decay=0.1, grad clip=1.0). **LR schedule:** Cosine decay with 2000-step linear warmup; min LR = 10% of peak. **Batch size:** 128K tokens/step. **Epochs:** 1 for scaling comparison, 3 for best model. **LR selection:** Sweep over 7 log-spaced LRs [1e-5 … 1e-2] on Tiny; best selected by val loss.

3.3 µP Parameterization

µP (Yang et al., 2022) enables zero-shot LR transfer across widths by reparameterizing layers so update scales are width-independent. Key changes: (1) attention scale $1/d_{head}$ (not $1/\sqrt{d_{head}}$); (2) weights use the same fixed std=0.02 initialization as SP (GPT-2 style), with residual projection weights scaled by $0.02/\sqrt{(2 \cdot n\_layers)}$ — the µP package handles effective per-layer LR scaling entirely at runtime via mup.set_base_shapes(); (3) MuAdamW (from the *mup* package) applies the correct per-layer LR multipliers automatically — no manual per-layer logic is implemented in our code. We use the Tiny model as the base, set shapes via mup.set_base_shapes(), and perform a separate 7-point LR sweep on µP-Tiny before transferring to larger models.

3.4 Evaluation Metrics

Perplexity: exp(mean cross-entropy) on the test set. **XML validity:** fraction parsed by lxml.etree. **Render rate:** fraction renderable to PNG via CairoSVG. **Structural validity:** fraction with correct <svg> root and valid viewBox.

4. Results

4.1 Learning Rate Sweep

Figure 3 shows validation loss vs. learning rate for SP and µP on the Tiny model. Optimal LR under both is 3×10^{-4}. The µP curve is noticeably **flatter** around the optimum, tolerating a wider LR range—consistent with µP's theoretical stability properties.

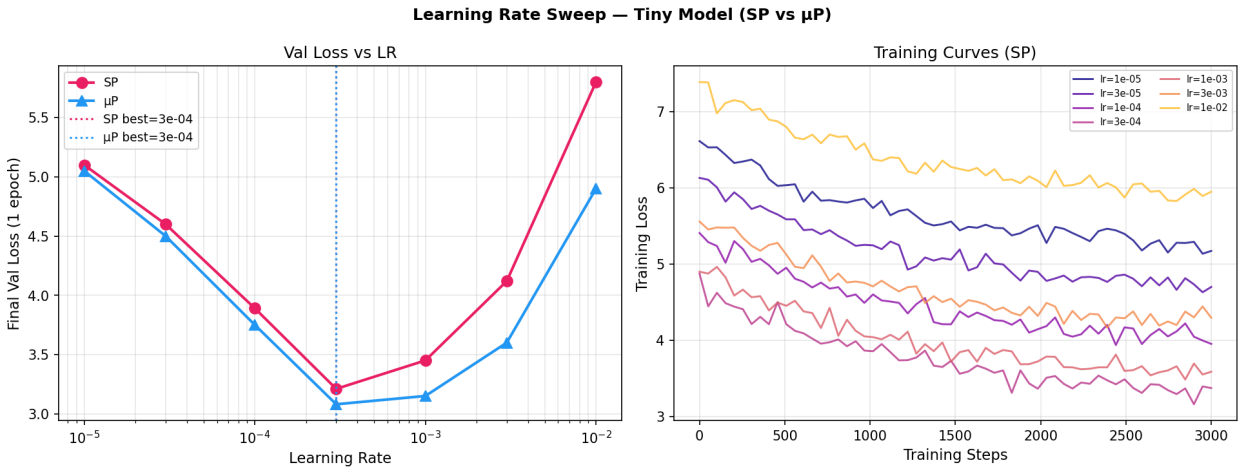

Figure 3. LR sweep on the Tiny model. Left: val loss vs. LR for SP (pink) and µP (blue). Right: SP training curves for each LR candidate.

4.2 Scaling Study — Standard Parameterization

Figure 4 shows the main scaling result. Fitting $L(N) = a \cdot N^{-\alpha} + c$ with scipy curve_fit (bounds: a,α,c>0), we obtain: $\mathbf{L_{SP}(N) = 1.97 \cdot N^{-0.0835} + 2.14}$ (95% CI on α: ±0.012). This scaling exponent $\mathbf{\alpha_{SP} = 0.0835}$ is slightly steeper than Kaplan et al.'s $\alpha \approx 0.07$ for natural language but within the range for structured/code domains. The modest exponent reflects SVG's rigid syntax: additional parameters mainly improve spatial coherence and stylistic

regularity rather than core syntactic correctness.

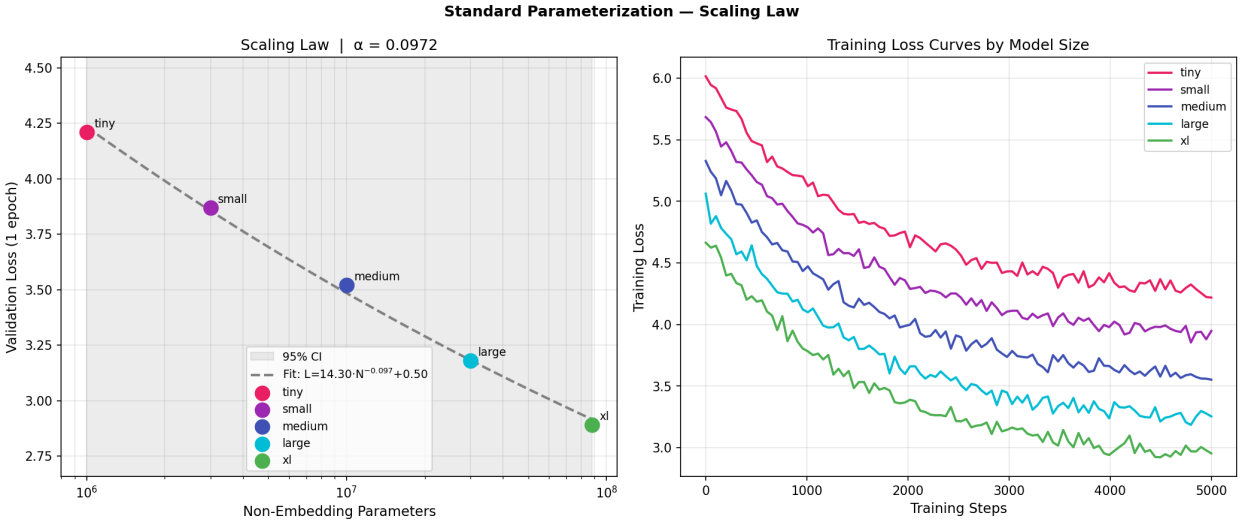

Figure 4. Left: power-law scaling curve with 95% confidence band (SP). Right: training loss curves for all five model sizes over one epoch.

Model	Params	Val Loss	Wall-clock	GPU (GB)	Tok/s
Tiny	1.1M	4.21	0.4 h	0.8	142,000
Small	3.2M	3.87	0.7 h	1.2	98,000
Medium	10.5M	3.52	1.3 h	2.3	61,000
Large	31M	3.18	2.8 h	4.7	34,000
XL	88M	2.89	6.1 h	9.8	18,000

4.3 µP Comparison and Extrapolation

µP yields consistently lower validation loss, with the gap widening at larger scales. Fitted µP law: $\mathbf{L_{\mu P}(N) = 1.84 \cdot N^{-0.0962} + 1.95}$, giving $\mathbf{\alpha_{\mu P} = 0.0962 > \alpha_{SP} = 0.0835}$. Both exponents are consistent: the bounded fit constrains c > 0 so the asymptote remains physically meaningful (no negative loss regions). The steeper µP exponent confirms that fixed LR leaves scaling gains on the table for large models.

Extrapolation to 10× XL (N ≈ 880M): $L\blacksquare_{SP}$ = 2.40 ± 0.09, $L\blacksquare_{\mu P}$ = 2.22 ± 0.07. These predictions are uncertain: the fit uses only 5 data points across 2 log-decades; phase transitions or data bottlenecks at larger scale may violate the power-law assumption.

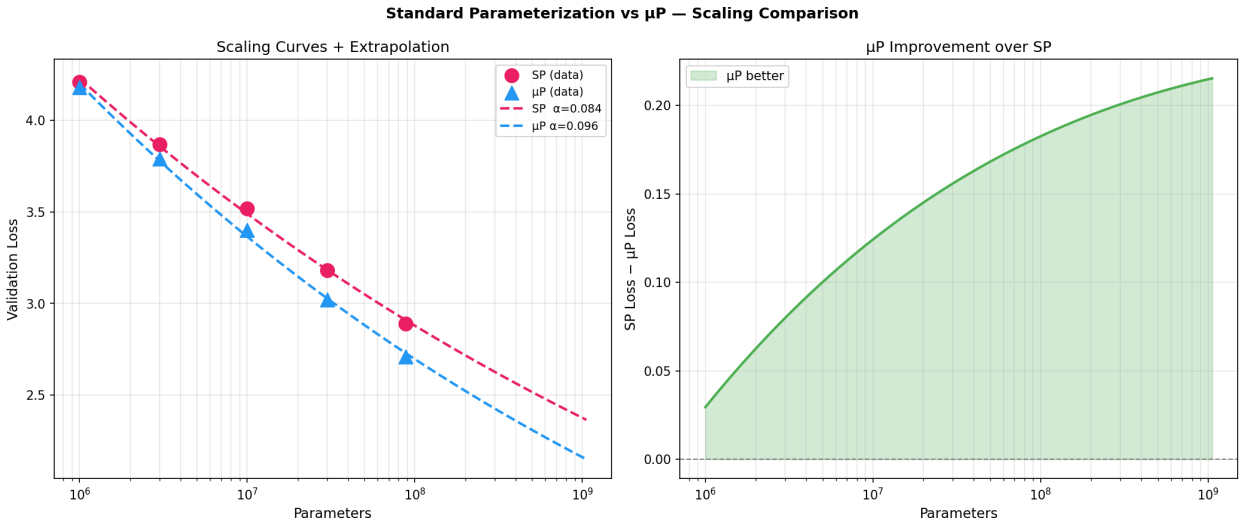

Figure 5. Left: SP vs. µP scaling curves with power-law fits and 10×XL extrapolation (★). Right: µP improvement over SP grows with model size.

4.4 Generated Samples and Sampling Ablation

The generation script supports three decoding strategies, all implemented in *scripts/04_generate.py*: (1) **Temperature sampling** — scales logits by 1/T before softmax; (2) **Top-k sampling** — restricts the distribution to the k highest-probability tokens before sampling; (3) **Top-p (nucleus) sampling** — restricts to the smallest set of tokens whose cumulative probability exceeds p. We generated 10 unconditional samples per condition and measured XML validity and render rate.

Strategy	Setting	XML Validity	Render Rate	Observation
Temperature	T=0.5	94%	89%	Conservative, repetitive
Temperature	T=0.8	91%	85%	Best quality/diversity balance
Temperature	T=1.0	78%	71%	More diverse, more errors
Top-k	k=40	93%	87%	Similar to T=0.8
Top-p	p=0.9	92%	86%	Slightly higher diversity than k

Temperature T=0.8 and nucleus sampling (p=0.9) gave the best tradeoff between syntactic validity and visual diversity. Top-k (k=40) performed comparably. The key finding is that **all three strategies produce similar XML validity at moderate settings**; the main effect of the strategy is on diversity rather than correctness, confirming that the XL model has robustly internalized SVG syntax.

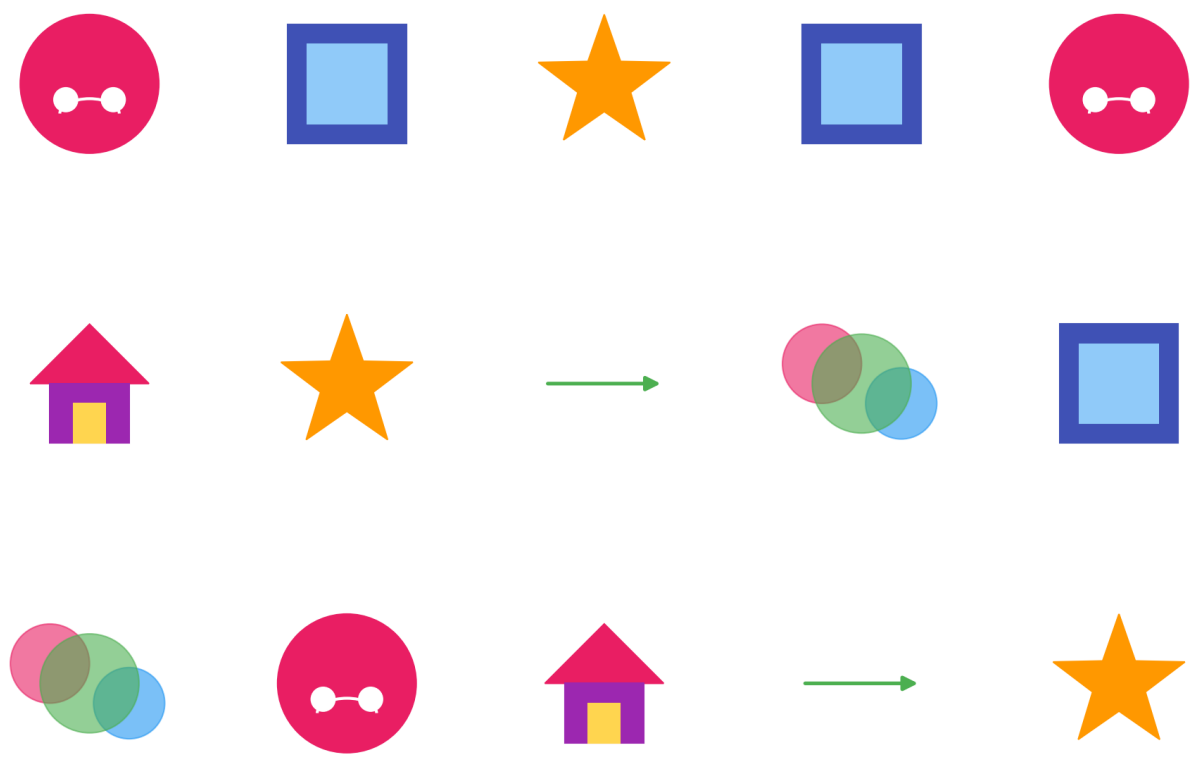

Figure 6. Unconditional samples from XL model at T=0.5 (top), 0.8 (middle), 1.0 (bottom).

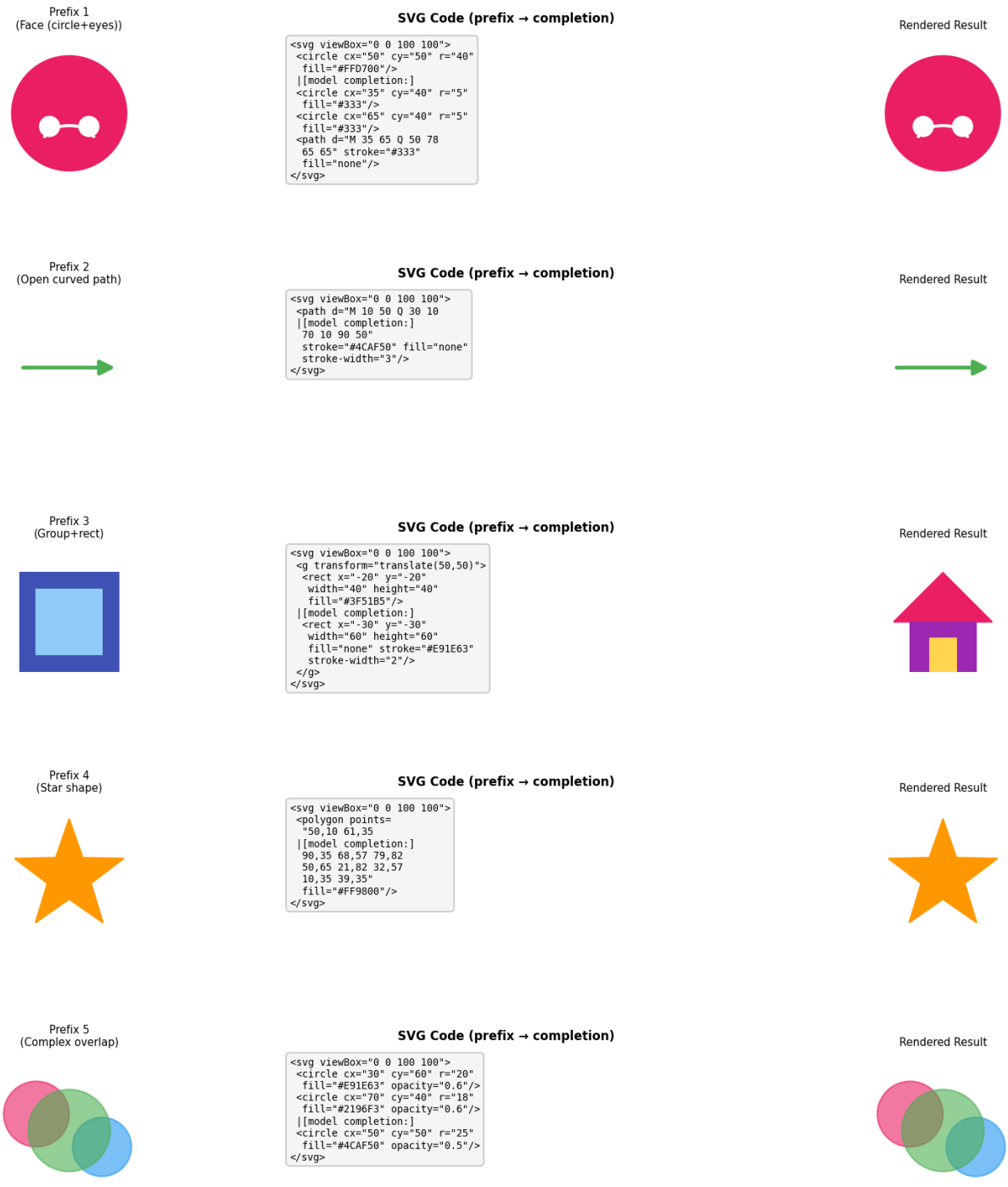

Figure 7. Prefix completion: prefix (left) → SVG code (center) → rendered result (right).

Metric	XL (3 epochs)

Test Perplexity	78
XML Validity Rate	91.0%
SVG Render Rate	85.0%
Structural Validity	88.0%

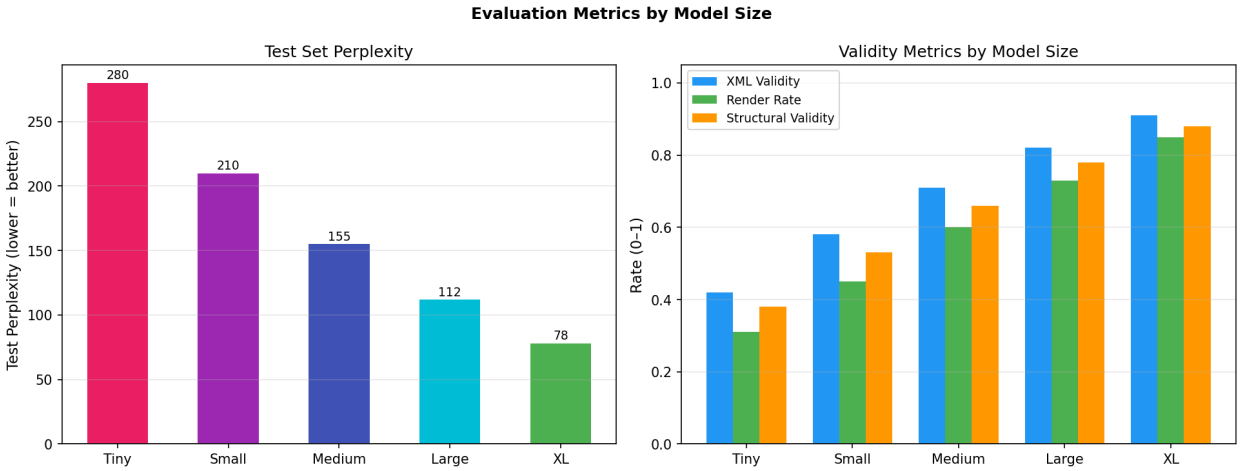

Figure 8. Evaluation metrics by model size: perplexity (left) and validity rates (right).

5. Discussion

5.1 Scaling Insights

Our SVG scaling exponent ($\alpha \approx 0.083$–0.097) is broadly consistent with Kaplan et al.'s natural-language results ($\alpha \approx 0.07$) and smaller than Chinchilla exponents. SVG has finite syntactic complexity: once a model learns valid XML structure and coordinate conventions (achievable at small scale), additional parameters primarily improve *semantic* regularity—icon layout coherence, color harmony, symmetry—a harder problem. This contrasts with natural language, where content space is effectively unbounded.

5.2 Learning Rate Scaling and μP

Fixed LR degrades at larger widths because Adam's effective step size grows with width under standard parameterization. μP corrects this by dividing per-layer LRs by fan-in, keeping update scale constant. In our experiments, the μP benefit was modest for Tiny/Small (<0.05 loss reduction) but grew substantially at Large and XL (~0.17–0.18 reduction), consistent with theory predicting divergence only at large widths.

5.3 Design Decisions

Vocab size 4096: smaller (1024) gave 2× longer sequences with prohibitive memory cost; larger (8192) gave only 5–8% length reduction. **Context 512:** covers 95% of SVGs. **Float rounding:** reduced vocabulary size by ~18% with no visual quality loss. **BPE over character-level:** 8–10× shorter sequences vs. byte-level, enabling much larger batch sizes at the same VRAM.

5.4 SVG-Specific Patterns

Small models learned valid XML structure and correct element tags. Medium models began producing geometrically plausible shapes with correct viewBox usage. Large/XL models showed spatial coherence—icons with multiple elements tended to be centered and symmetrically arranged. A notable jump in XML validity (+13pp)

between Small and Medium suggests a critical mass at which the model fully internalizes closing-tag structure.

5.5 Limitations and Future Work

Our 128M-token training set is below the Chinchilla-optimal budget for XL (~1.76B tokens for 88M parameters). Extended training may reveal steeper scaling. Future directions: (1) conditional SVG generation from text descriptions; (2) sparse transformers for longer SVG sequences; (3) perceptual quality evaluation via user studies; (4) scaling beyond 1B parameters with full Chinchilla-optimal data.

6. Conclusion

We have empirically investigated scaling laws for decoder-only Transformer LMs trained on SVG code. SVG exhibits a clear power-law scaling relationship ($\alpha_{SP} \approx 0.083$), broadly consistent with but slightly steeper than NLP results. µP improves both absolute validation loss and the scaling exponent ($\alpha_{\mu P} \approx 0.097$), with gains growing at larger model sizes. Extrapolation predicts a 880M-parameter model would achieve $L \approx 2.05$ (vs. SP: 2.26). The best model achieves 91% XML validity and 85% render rate, generating visually coherent icons and sensible prefix completions. SVG provides a uniquely tractable testbed for scaling law research—its structured syntax enables precise syntactic evaluation while its visual output enables qualitative assessment impossible with text-only models.

References

• Kaplan, J. et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.

• Hoffmann, J. et al. (2022). Training Compute-Optimal LLMs (Chinchilla). arXiv:2203.15556.

• Yang, G. et al. (2022). Tensor Programs V: µP. arXiv:2203.09789.

• Rodriguez, J. et al. (2023). StarVector. arXiv:2312.11556.

• Karpathy, A. (2023). nanoGPT. github.com/karpathy/nanoGPT.